A computer system has four different types of resources (Printer, ROM, Hard Disk and RAM) and it needs to complete execu on of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execu on and total available resources should be taken from the user. Write a program to: • Calculate current need of resources by each process. • Find whether these processes can be executed with available resources. • If yes, show the correct order of process execu on

CODE:-

```
  GNU nano 8.7                                                    banker1.c
#include <stdio.h>

#define P 5   // Number of processes
#define R 4   // Number of resources

int main() {

    int allocation[P][R] = {
        {0,0,1,4},   // P0 Google Drive
        {0,6,3,2},   // P1 Firefox
        {0,0,1,2},   // P2 Word Processor
        {1,0,0,0},   // P3 Excel
        {1,3,5,4}    // P4 PowerPoint
    };

    int max[P][R] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[R] = {1,6,2,0};

    int need[P][R];
    int finish[P] = {0};
    int safeSeq[P];
    int work[R];

    int i, j, k;

    // Step 1: Calculate Need Matrix
    printf("Need Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Copy available to work
    for(i = 0; i < R; i++)
        work[i] = available[i];
```

```c
    int count = 0;

    // Step 2: Banker's Algorithm
    while(count < P) {
        int found = 0;

        for(i = 0; i < P; i++) {
            if(finish[i] == 0) {

                for(j = 0; j < R; j++) {
                    if(need[i][j] > work[j])
                        break;
                }

                if(j == R) {

                    for(k = 0; k < R; k++)
                        work[k] += allocation[i][k];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("System is NOT in SAFE state.\n");
            return 0;
        }
    }

    // Step 3: Print Safe Sequence
    printf("\nSystem is in SAFE state.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < P; i++)
        printf("P%d ", safeSeq[i]);

    return 0;
}
```

OUTPUT:-

```
nn
VIDISHA@DESKTOP-KHV52LE MINGW64 ~/OneDrive/Desktop
$ nano banker1.c

VIDISHA@DESKTOP-KHV52LE MINGW64 ~/OneDrive/Desktop
$ gcc banker.c -o banker

VIDISHA@DESKTOP-KHV52LE MINGW64 ~/OneDrive/Desktop
$ gcc banker1.c -o banker1

VIDISHA@DESKTOP-KHV52LE MINGW64 ~/OneDrive/Desktop
$ ./banker1
Need Matrix:
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

System is in SAFE state.
Safe Sequence: P1 P2 P3 P4 P0
VIDISHA@DESKTOP-KHV52LE MINGW64 ~/OneDrive/Desktop
$
```